Adevar Labs

Prepared for:

Reflect Money

October 08, 2026

Proxy Oracle Stacking Audit Report

Version: after fix review

Table of Contents

1. Executive Summary 4

About Proxy Oracle Stacking 4

Audit Summary 4

Overall Security Posture 4

After Fix Review Summary 4

Before Fix Review Summary 5

Launch Recommendations 5

Audit Scope 5

2. Assumptions and Considerations 6

Audit Assumptions 6

3. Severity Definitions 7

Impact 7

Likelihood 7

Severity Classification Matrix 7

4. Findings 9

M01: An empty pull account skips the price check for the whole chain 9

M02: A permissionless Pyth post can replace a newer price 11

L01: Staleness bounds counted in slots and documented at 400 ms/slot go stale hours early at today's slot rate 13

L02: wrap has no undercollateralization guard, so a deposit into an underwater vault overpays for shares and is locked 14

L03: Wrap, post a newer lower price, and unwrap in one transaction drains unclaimed integrator commission 18

L04: Two Assets on one feed can store a book the tighter Asset cannot read 20

L05: Two Assets on one feed can flip the stored scale 22

L06: A future observationsTimestamp can freeze a Data Streams feed 24

© 2026 Adevar Labs. All rights reserved.

L07: Opposite inverse flags on one feed compose to $1.00 26

5. Enhancement Opportunities 27

E01: claim_fees takes no signer, so any caller picks the price at which commission settles 27

E02: SDKs still encode the pre-PR Asset layout and instruction data 29

E03: The per-leg confidence bound check runs against non-Pyth legs 30

E04: Chainlink staleness checks slot and skips Transmission.timestamp 31

About Us 32

About Adevar Labs 32

Audit Methodology 32

Confidentiality Notice 34

Legal Disclaimer 34

© 2026 Adevar Labs. All rights reserved.

1. Executive Summary

About Proxy Oracle Stacking

Reflect Proxy wraps stablecoins into a branded token. Each Asset is priced from an oracle chain. This is a review of pull request 26, **feat/oracle-stacking**.

Audit Summary

The table below summarizes identified vulnerabilities by severity and remediation status.

Severity	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	2	2	0
Low	7	7	0

Enhancement opportunities: 4.

Overall Security Posture

We found no Critical or High issues. The stacking path is in place: a chain of legs, a stored pull price, and a second check on read. Two issues are rated Medium. One skips the price band when a pull account is empty. The other lets a Pyth post replace a newer price. Seven issues are rated Low, and there are four enhancement opportunities.

After Fix Review Summary

The Reflect team fixed both Medium issues, all seven Low issues, and three of the four enhancements. The fourth, the SDK update, is done on a branch that has not yet been merged.

A chain with an unposted pull feed now still checks every other leg, and the Asset cannot be used until the whole chain has passed the price band. A Pyth post must now be newer than the stored price. Each Asset keeps its own Data Streams price, so Assets that share a feed no longer affect each other. **wrap** now refuses an undercollateralized vault, a price drop can no longer be paid out of the integrator's unclaimed commission, and **claim_fees** now requires the proxy authority to sign.

© 2026 Adevar Labs. All rights reserved.

Staleness bounds are now kept in seconds. Doppler publishes no timestamp, so a Doppler leg is still bounded in slots and expires somewhat early at today's slot rate.

Before Fix Review Summary

We reviewed oracle stacking. An Asset can name several legs. More than one Asset can name the same feed. **post_oracle_price** writes one account, and wrap, unwrap, and fee claims read it back.

We checked the providers in that path: Pyth, Chainlink Data Streams v3, v7 rates, Doppler, and Data Feeds. We checked straight and inverse legs, shared feeds, confidence bounds, staleness, and the price band. We also checked a new report posted in the same transaction as a wrap or an unwrap.

Two issues are rated Medium, seven are rated Low, and four are enhancement opportunities. Details are in the Findings section.

Launch Recommendations

- Set a tight **min** and **max** around the real composed price. A 10x band does not catch a flipped **inverse** near $1.
- After creating an Asset with a pull feed, post every feed and run **update_oracle** again. The Asset refuses every read until its whole chain has been priced against the band.
- Re-save the oracle chain of any Asset set up with the earlier pull request 26 code. Staleness bounds are now read in seconds, and Data Streams prices now live at a per-Asset address.
- Size Doppler staleness bounds from the measured slot time, since Doppler is still bounded in slots.
- Merge the regenerated SDKs before integrators use the new Asset layout.
- Keep the USDC+ chain on the same price as swap. Deactivation does not stop wraps.

Audit Scope

- **Repository:** https://github.com/palindrome-eng/reflect-proxy-program
- **Pull request:** https://github.com/palindrome-eng/reflect-proxy-program/pull/26 (**feat/oracle-stacking**)
- **Before Fix Review Commit Hash:** **8b71e4efd33a72e1a37af6397aea8e05f4be4414**
- **After Fix Review Commit Hash:** **3350057617db4f9b30afe449db5db8c952423e68**
- **Files/Modules in Scope:**
  - **program/src/**

© 2026 Adevar Labs. All rights reserved.

2. Assumptions and Considerations

Audit Assumptions

1. A 10x price band cannot detect a wrong inverse on a rate near 1.0

create_asset and **update_oracle** accept a live composed price only when **min <= price <= max** and **max <= 10 x min**. That check catches a wrong hop whose price is far from the intended one (BTC in place of USDC). It does not catch a flipped **inverse** bit on a feed near 1.0 (USDC/USD, syrupUSDC/USDC): **r** and **1/r** both sit inside any band that wide. The program states this. Only **AUTHORITY_ID** sets the bit. Correct direction is the authority's responsibility. This review treats a well-set chain as the intended case.

2. Swap stays on the hardcoded Doppler **ORACLE_ID**

Swap continues to use the hardcoded, 17-byte Doppler oracle at **ORACLE_ID**. Wrap, unwrap, and fee claims instead value each Proxy's backing Asset through its configured oracle chain. The cloned production USDC+ Asset currently points to **ORACLE_ID**, but this PR does not require future updates to that Asset's chain to match the swap oracle. If the two prices diverge, the paths could value USDC+ differently. That is a cross-path consistency risk to consider when changing the USDC+ Asset configuration; the split alone is not a demonstrated defect.

3. Deactivation prevents new proxies but not new wraps

deactivate_asset sets the **active** flag on the Asset to zero. **create_proxy** reads this flag and refuses to create a new proxy on an inactive asset. **wrap**, **unwrap**, and **claim_fees** do not read the flag.

Existing proxies on a deactivated asset therefore continue to accept new deposits, redemptions, and fee claims. The design intent, documented in **deactivate_asset.rs**, is to keep exits open for existing users. No check separates **wrap** from **unwrap**, so keeping exits open also keeps deposits open.

An operator who deactivates an asset over an oracle concern should tell integrators that new wraps still succeed. Each integrator must freeze their own proxy through **FreezeFee** or halt user access off-chain. No protocol-level wrap-pause primitive exists.

4. The admin and the oracle signers are trusted

The proxy authority sets the chain, the **inverse** bit, the decimals, the confidence bound, and the staleness window. Pyth and the Chainlink DON sign the reports this program stores. This review assumes that the admin is honest and careful, and that those signers publish the price they claim to publish. A bad chain, a flipped **inverse**, or a wrong scale set by that admin is a centralization risk, not a defect in the program. A signer that attests to a false price is the same.

© 2026 Adevar Labs. All rights reserved.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help convey both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements, typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product's reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue, particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

- **High Impact**: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact**: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact**: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood**: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood**: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood**: Exploitation of these vulnerabilities is often technically infeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

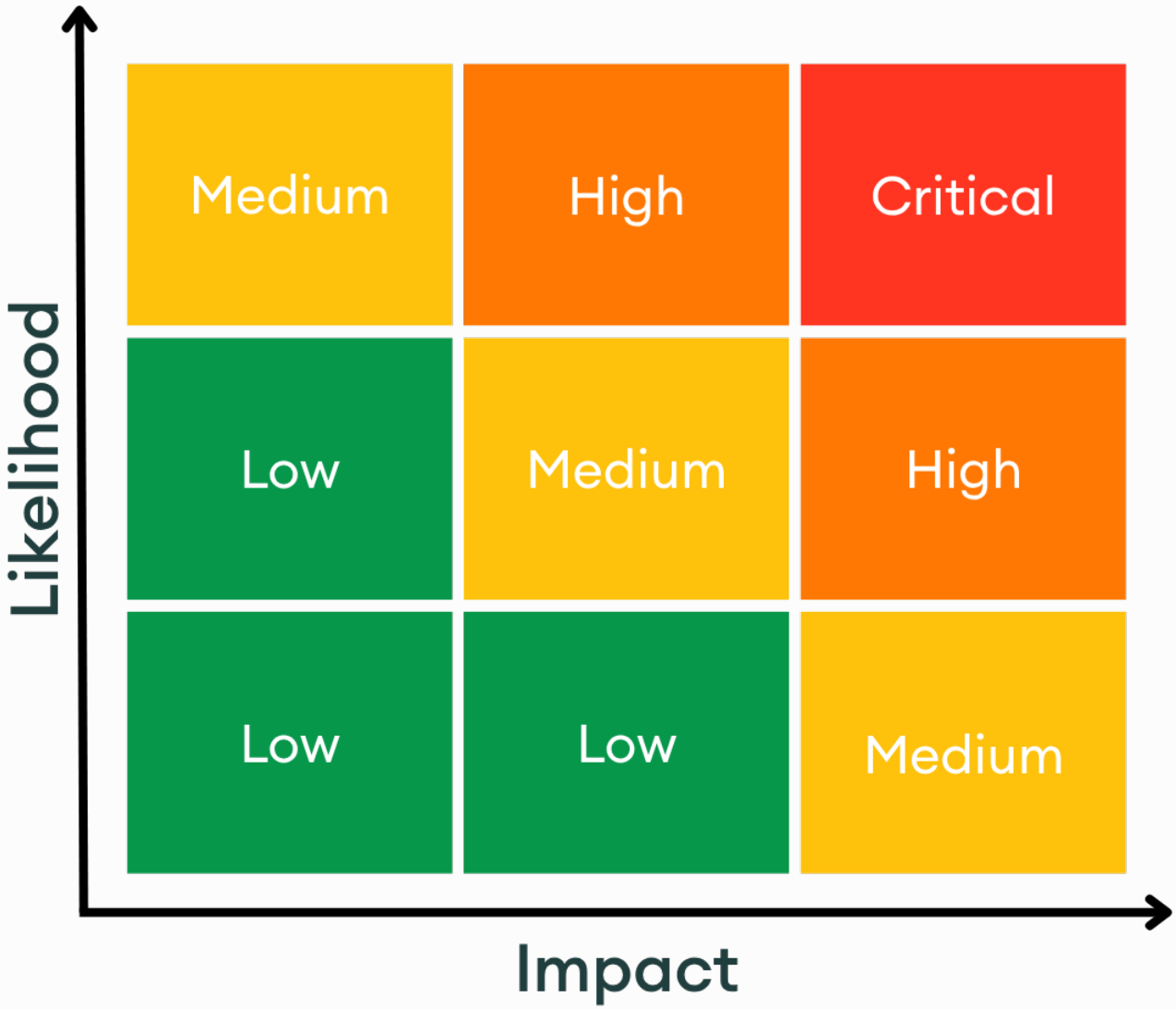
By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical**: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High**: High impact with medium likelihood, or medium impact with high likelihood
- **Medium**: Moderate impact and/or discoverability

© 2026 Adevar Labs. All rights reserved.

- **Low**: Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

© 2026 Adevar Labs. All rights reserved.

4. Findings

M01: An empty pull account skips the price check for the whole chain



LOCATION

reflect-proxy-program/9819f6e6/program/src/proxy/update_oracle.rs:L325-L332

RELEVANT SNIPPET

>_ update_oracle.rs — RUST

```
323: // and nothing wider: a Pyth leg on Pyth's own sponsored account resolves as it always
324: // did, because that account exists.
325: let unposted = self.names_an_unposted_pull_leg()?;
326:
327: let composed = if unposted {
328:     None
329: } else {
330:     let asset = Asset::load(self.accounts.asset)?;
331:     Some(asset.resolve_price(self.accounts.new_oracle, self.accounts.extra_oracles)?)
332: };
333:
334: // A wrong leg order or a wrong feed moves the composed price by orders of magnitude,
```

DESCRIPTION

A pull feed has to be saved while its account is still empty, because **post_oracle_price** will not create that account until an Asset names it. **update_oracle** does this by setting **composed = None**, which skips **resolve_price** for every leg.

The price band, **min_price_1e6** and **max_price_1e6**, is only an argument of this instruction. It is not stored, and **wrap** never reads it. The band runs only when **composed** is **Some**, so the first configuration of a pull feed never checks the price.

A Data Streams report does not say whether its price is 8 or 18 decimals. Both are allowed. Setting 8 on an 18-decimal stream makes the asset look about **10^10** too expensive, and that is the direction in which **wrap** mints too much. The band would reject that, but this path does not run the band.

One empty Pyth or Data Streams leg is enough to skip the others too. A Doppler or Data Feeds leg in the same chain is stored without an owner, feed, or price check. A Pyth account that already exists is still checked.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

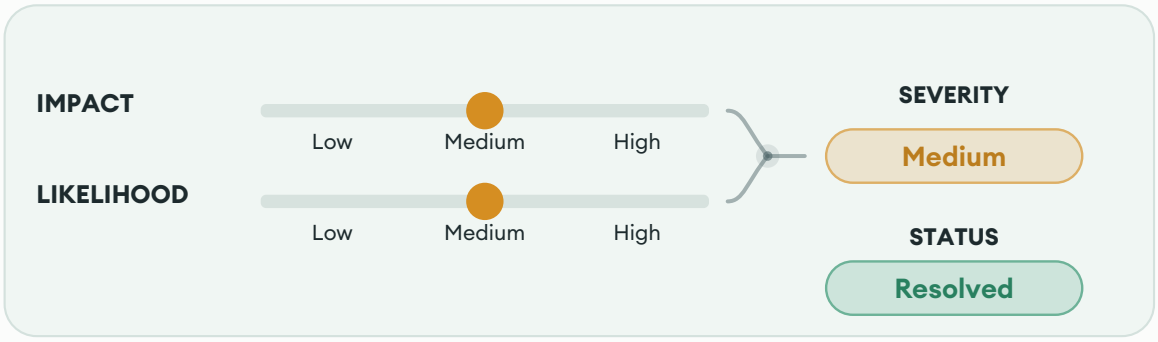
Keep the first **update_oracle** able to store the chain while the pull account is empty. On that call, still load every leg whose account already exists. Store the band on the Asset, and run it from **resolve_price** once every leg can be read, so **wrap**, **unwrap**, **swap**, and **claim_fees** all see it.

FIX REVIEW NOTES

Fixed in **c5a1de9**. When a pull feed has not been posted yet, **update_oracle** now still loads and checks every leg whose account exists. An Asset whose chain has never been fully priced is refused on every read until a later **update_oracle** prices the whole chain against the band.

© 2026 Adevar Labs. All rights reserved.

M02: A permissionless Pyth post can replace a newer price



LOCATION

reflect-proxy-program/9819f6e6/program/src/proxy/post_oracle_price.rs:L598-L599

RELEVANT SNIPPET

>_ post_oracle_price.rs — RUST

```
596:     Seed::from(seeds[2]),
597: ];
598: let authority_bump = [PROTOCOL_AUTHORITY_BUMP];
599: let authority_seeds = [Seed::from(AUTHORITY_SEED), Seed::from(&authority_bump)];
600:
601: // The constant pins the CPI target. `post_update` takes the payer as a writable signe
       r,
```

DESCRIPTION

post_oracle_price is permissionless. The Pyth branch signs the CPI as **PROTOCOL_AUTHORITY**. The receiver stores that key as **write_authority**, so a later post needs this signature.

The receiver writes the new price and sets **posted_slot** to the current slot. It does not compare **publish_time**. **check_price_update** does not read **publish_time** either. It checks the owner, the discriminator, **write_authority**, full verification, and the feed id.

RUST

```
// program/src/proxy/post_oracle_price.rs
if data[PYTH_FEED_ID..PYTH_FEED_ID + 32].ne(feed_id) {
    return Err(ProxyError::InvalidOracleFeed.into());
}

Ok(())
```

load_pyth_price checks **posted_slot** and **publish_time** against the leg window. The new slot is current, so the slot check passes. A **publish_time** inside the window passes too.

A stored staleness of 0 selects 200 slots, about 80 seconds. The authority can set the leg up to 1,500 slots, about 10 minutes. The caller chooses any signed Pyth update inside that window.

The next instruction in the same transaction reads the account. **wrap**, **unwrap**, **swap**, and **claim_fees** use that price.

The Data Streams branch returns **OracleReportNotNewer** when the new observation is older than or equal to the stored one. The Pyth branch does not keep that rule.

© 2026 Adevar Labs. All rights reserved.

An update older than the window still posts. **load_pyth_price** then returns **StaleOracle**. The caller can replace a later fresh post with that older update.

PRACTICAL IMPACT

Any caller with a signed Pyth update from inside the window can post it and use it in one transaction. **wrap**, **unwrap**, **swap**, and **claim_fees** then move value at that price. An older update still posts, and later reads return **StaleOracle** until a newer post remains in the account. Worst case: the caller selects the worst signed price in the window and wraps or unwraps at it.

RECOMMENDATION

Before the CPI, read the stored **publish_time**. After the CPI, require the new **publish_time** to be greater. On the first write, accept only an update inside the leg window.

FIX REVIEW NOTES

Fixed in **b7ca56e**. A Pyth post now reads the stored **publish_time** first and is refused unless the new update is strictly newer and not ahead of the clock.

© 2026 Adevar Labs. All rights reserved.

L01: Staleness bounds counted in slots and documented at 400 ms/slot go stale hours early at today's slot rate



LOCATION

reflect-proxy-program/8b71e4ef/program/src/helpers.rs:L485

RELEVANT SNIPPET

>_ helpers.rs RUST

```
483: /// Seconds a slot is worth, as a fraction, for converting a slot bound into a time bo
       und.
484: /// Solana targets 400 ms per slot.
485: const SLOT_MILLIS: u64 = 400;
486:
487: /// The leg's staleness bound expressed in seconds.
```

OTHER LOCATIONS

- reflect-proxy-program/8b71e4ef/program/src/constants.rs:L210-L214

DESCRIPTION

A leg's staleness bound is stored in slots, sized from a documented **SLOT_MILLIS = 400** ms per slot ("48 hours" for the **ChainlinkPush** ceiling of 432,000 slots). Mainnet slots now pass faster: **getBlockTime** over the last 432,000 and 1,000,000 slots gives 0.265 s/slot, and fixture (slot, timestamp) pairs give 0.316 s/slot. The slot check binds before the derived seconds check, so a leg sized from the 400 ms figure goes stale hours earlier than the operator planned.

RECOMMENDATION

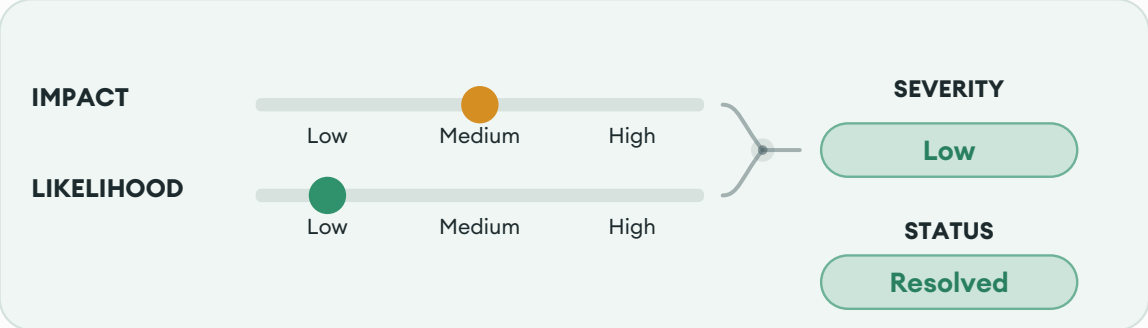
Size **ChainlinkPush** bounds from **Transmission.timestamp** in seconds rather than from a slot count, or document that operators must convert using the measured slot rate with margin, not the nominal 400 ms.

FIX REVIEW NOTES

Fixed in **52040ce**. Staleness bounds are now stored in seconds, and providers that publish a timestamp are judged by that timestamp, so they no longer go stale early when slots run faster than 400 ms. Doppler publishes no timestamp and is still bounded in slots, so a Doppler leg still expires before its nominal bound at today's slot rate (about 63 of 80 seconds at 0.316 s/slot). This errs toward refusing a price.

© 2026 Adevar Labs. All rights reserved.

L02: wrap has no undercollateralization guard, so a deposit into an underwater vault overpays for shares and is locked



LOCATION

reflect-proxy-program/8b71e4ef/program/src/state.rs:L1172-L1284

RELEVANT SNIPPET

>_ state.rs — RUST

```rust
/// Note: This helper function eliminates the need to repeatedly deserialize the Proxy State account
/// in both mutable and immutable contexts, reducing code duplication and improving maintainability.
pub fn wrap(
    &mut self,
    oracle_price: &OraclePrice,
    stablecoin_proxy_state_vault: &AccountInfo,
    branded_mint: &AccountInfo,
    branded_user_token_account: &AccountInfo,
    proxy_state: &AccountInfo,
    wrap_amount: u64,
    min_branded_tokens: u64,
) -> Result<(), ProgramError> {
    // The caller resolved the Asset's oracle chain (1..=3 legs) before entering here.
    let exponent = oracle_price.exponent;
    let oracle_price = oracle_price.price;

    // Calculate the USDC value of the Vault
    let vault_usdc_value = {
        let amount = TokenAccount::from_account_info(stablecoin_proxy_state_vault)?.amount();
        u64::try_from(
            mul_oracle(amount as u128, oracle_price, exponent)
                .ok_or(ProgramError::ArithmeticOverflow)?,
        )
        .map_err(|_| ProgramError::ArithmeticOverflow)?
    };

    // Crank the ProxyState
    self.crank(vault_usdc_value)?;

    // Calculate the amount of Branded Tokens to mint (with virtual offsets)
    // Read supply and decimals, then drop the Ref so branded_mint can be used by MintTo CPI later.
    let (branded_mint_supply, decimals) = {
        let m = Mint::from_account_info(branded_mint)?;
```

© 2026 Adevar Labs. All rights reserved.

>_ state.rs RUST

```rust
        (m.supply(), m.decimals())
    };

    // Virtual accounting offsets computed from token decimals to support arbitrary pr
ecision.
    // For 6-decimal tokens this equals 1_000_000, for 9-decimal tokens 1_000_000_000,
 etc.
    let virtual_offset = 10u64.pow(decimals as u32);

    // Convert wrap_amount (USDC+ tokens) to USDC value to match principal units
    let wrap_amount_usdc_value = u64::try_from(
        mul_oracle(wrap_amount as u128, oracle_price, exponent)
            .ok_or(ProgramError::ArithmeticOverflow)?,
    )
    .map_err(|_| ProgramError::ArithmeticOverflow)?;

    // shares_to_mint = wrap_amount_usdc_value * (supply + virtual_offset) / (principa
l + virtual_offset)
    let denom = self
        .principal()
        .checked_add(virtual_offset)
        .ok_or(ProgramError::ArithmeticOverflow)?;

    let numer_multiplier = branded_mint_supply
        .checked_add(virtual_offset)
        .ok_or(ProgramError::ArithmeticOverflow)?;

    let amount = u64::try_from(
        (wrap_amount_usdc_value as u128)
            .checked_mul(numer_multiplier as u128)
            .and_then(|x| x.checked_div(denom as u128))
            .ok_or(ProgramError::ArithmeticOverflow)?,
    )
    .map_err(|_| ProgramError::ArithmeticOverflow)?;

    // Ensure the mint amount is greater than 0 to prevent user loss
    if amount == 0 {
        return Err(ProxyError::InvalidAmount.into());
    }

    // Slippage protection: ensure the calculated amount meets the minimum threshold
    if amount.lt(&min_branded_tokens) {
        return Err(ProxyError::SlippageExceeded.into());
    }

    // Deposit cap: ensure new principal won't exceed the cap (0 = unlimited)
    let cap = self.deposit_cap();
    if cap > 0 {
        let new_principal = self
            .principal()
            .checked_add(wrap_amount_usdc_value)
            .ok_or(ProgramError::ArithmeticOverflow)?;
        if new_principal > cap {
            return Err(ProxyError::DepositCapExceeded.into());
        }
    }
```

© 2026 Adevar Labs. All rights reserved.

>_ state.rs RUST

```
1256:
1257:     // Mint the Branded Tokens to the user based on the amount of USDC+ deposited
1258:     let seeds = [
1259:         Seed::from(PROXY_STATE_SEED),
1260:         Seed::from(branded_mint.key().as_ref()),
1261:         Seed::from(self.bump()),
1262:     ];
1263:
1264:     MintToChecked {
1265:         mint: branded_mint,
1266:         account: branded_user_token_account,
1267:         mint_authority: proxy_state,
1268:         amount,
1269:         decimals,
1270:     }
1271:     .invoke_signed(&[Signer::from(&seeds)])?;
1272:
1273:     // Log the Wrap Event
1274:     log_wrap_event(
1275:         proxy_state.key(),
1276:         branded_user_token_account.key(),
1277:         wrap_amount,
1278:         amount,
1279:     );
1280:
1281:     // Add the principal to the proxy_state
1282:     // Use the USDC value of the wrapped amount (already calculated above)
1283:     self.add_principal(wrap_amount_usdc_value)
1284: }
1285:
1286: /// # Unwrap
```

OTHER LOCATIONS

- reflect-proxy-program/8b71e4ef/program/src/state.rs:L1328

DESCRIPTION

unwrap and **claim_fees** both refuse when **vault_usdc_value < principal()**, that is, when the vault's real value has fallen below what it owes depositors. **wrap** carries no such check. After a price drop leaves the vault underwater, **wrap** still mints **wrap_amount_usdc_value*(supply+off)/(principal+off)**, pricing new shares against the stale, too-high **principal**. The deposit is locked immediately by the same **unwrap** guard, and when the price recovers **crank** books the rebound as fresh interest: the fee goes to commission and the remainder spreads across all shares, mostly the earlier depositors'.

RECOMMENDATION

Apply to **wrap** the same guard that **unwrap** and **claim_fees** use.

RUST

```
if vault_usdc_value < self.principal() {
    return Err(ProxyError::VaultUndercollateralized.into());
}
```

© 2026 Adevar Labs. All rights reserved.

FIX REVIEW NOTES

Fixed in **9399876**. **wrap** now refuses with **VaultUndercollateralized** when the vault value is below principal, the same guard **unwrap** and **claim_fees** use.

© 2026 Adevar Labs. All rights reserved.

L03: Wrap, post a newer lower price, and unwrap in one transaction drains unclaimed integrator commission



LOCATION

reflect-proxy-program/8b71e4ef/program/src/state.rs:L1106-L1114

RELEVANT SNIPPET

>_ state.rs — RUST

```
1104:     .ok_or(ProgramError::ArithmeticOverflow)?;
1105:
1106: let interested_generated = if usdc_value >= total_principal_and_commission {
1107:     usdc_value
1108:         .checked_sub(total_principal_and_commission)
1109:         .ok_or(ProgramError::ArithmeticOverflow)?
1110: } else {
1111:     // When oracle price decreases, vault value is less than recorded principal + comm
      ission
1112:     // In this case, no new interest is generated
1113:     0
1114: };
1115:
1116: // Split interest: compute commission first (floor), give dust to principal
```

OTHER LOCATIONS

- reflect-proxy-program/8b71e4ef/program/src/state.rs:L1328-L1370

DESCRIPTION

ProxyState records principal in USDC and pays it back in tokens at the current oracle price. The integrator's unclaimed commission is whatever the vault holds above that principal.

1. **crank** books no loss when the price falls.
2. **unwrap** pays **div_oracle(user_share_usdc_value, price)**, so a lower price pays more tokens for the same USDC principal.
3. The **unwrap** guard only requires **vault_usdc_value >= principal**, not principal plus commission.

A price drop is therefore absorbed by the unclaimed commission. **post_oracle_price** is permissionless and accepts any report newer than the stored one, including a lower one. A caller can **wrap** at the stored price, post a newer lower report, and **unwrap** in one transaction. The extra tokens paid on the caller's own principal come out of the commission, up to all of it. After the round trip the vault sits exactly at principal, so a further dip blocks every **unwrap** with **VaultUndercollateralized**.

© 2026 Adevar Labs. All rights reserved.

The trigger is any Asset whose leg can post a newer, lower price: Pyth or Data Streams price feeds. Requiring newer reports (M02) does not stop it.

PROOF OF CONCEPT

A Mollusk test runs the compiled program at **8b71e4e**, with the suite's mock Data Streams verifier standing in for DON signatures. All four instructions share one account set and one clock.

Full test and run steps: https://gist.github.com/ArsenSecurity/a58491b781ee19c794ff14d2416855d0

Setup:
- Vault holds 1,000,000 USDC+ at $1.0500
- Principal $1,045,000, unclaimed commission $5,000, fee 10%
- Attacker holds 9,000,000 USDC+

Attack:
1. **wrap** 9,000,000 at $1.0500. Principal grows by $9,450,000.
2. **post_oracle_price** with a newer report at $1.0495 (a 4.8 bp dip). It passes **OracleReportNotNewer**.
3. **unwrap** all shares. The attacker receives $9,450,000 / 1.0495 = 9,004,287.76 USDC+.
4. **claim_fees** pays the integrator 0.

Control, same dip with no attacker: **claim_fees** pays 4,287.76.

```
control: integrator claims                4287.756074 USDC+
attack : attacker profit                  4287.756074 USDC+
attack : integrator claims afterwards        0.000000 USDC+
```

RECOMMENDATION

Cap each **unwrap** at the caller's pro-rata share of the vault tokens not reserved for commission, so a price move cannot pay one exiter out of the integrator's balance.

RUST

```rust
let commission_tokens = div_oracle_ceil(self.integrators_commission() as u128, oracle_price, e
  xponent)?;
let cap = unwrap_amount as u128 * (vault_amount - commission_tokens) as u128 / branded_supply
  as u128;
let amount = amount.min(cap);
```

The same run through the capped formula leaves the attacker 2.04 USDC+ short and the integrator claiming 4,289.80. Depositors who exit during a real dip then bear their own share of it.

FIX REVIEW NOTES

Fixed in **98990fa**. **wrap** and **unwrap** now price shares against the smaller of principal and the vault value less unclaimed commission, so a price drop is borne by holders rather than paid from the commission. **claim_fees** now reduces the commission by the value it pays instead of zeroing it.

© 2026 Adevar Labs. All rights reserved.

L04: Two Assets on one feed can store a book the tighter Asset cannot read



LOCATION

reflect-proxy-program/8b71e4ef/program/src/helpers.rs:L949-L951

RELEVANT SNIPPET

>_ helpers.rs — RUST

```
947: }
948:
949: if confidence_bps > confidence_bound_bps(max_confidence_bps) {
950:     return Err(ProxyError::OracleConfidenceTooWide.into());
951: }
952:
953: Ok(OraclePrice {
```

DESCRIPTION

One Chainlink v3 feed has one stored price. The account is a PDA derived from the seed **stream** and the feed id. Every Asset that names that feed reads that same account.

The keeper builds **post_oracle_price** and includes one Asset account. The program reads the confidence limit from the first leg whose oracle address is that account. It does not read the other Assets that share the feed.

Use two Assets on one feed. Both declare the same decimals. Tight allows a 2% spread. Loose allows a 10% spread. A signed report has a 5% spread. A post through Tight fails. A post through Loose succeeds. The saved spread is 5%.

Tight then reads the same account. 5% is above its 2% limit, so the read fails. An older report that was inside 2% cannot replace the saved one, because **post_oracle_price** only accepts a report newer than the stored one. Tight has no price until a later report is both newer and inside 2%.

If the admin sets no limit, the program uses 2%. The highest an admin can set is 10%. The report must still pass the limit of the Asset in the transaction.

Sharing one feed is allowed. The nine cloned production Assets do not share a v3 feed. Three name one Doppler oracle, and Doppler publishes no spread. Each active Pyth Asset has its own feed. If both Assets use the same limit, a report one accepts is a report the other accepts. A v7 rate report stores no spread, so this path does not apply to a redemption-rate leg.

© 2026 Adevar Labs. All rights reserved.

RECOMMENDATION

Do not let one Asset overwrite the shared price with a report another Asset on that feed cannot read. Key the price account by Asset as well as feed id, or require the saved report to pass every Asset that names the feed.

FIX REVIEW NOTES

Fixed in **448213c**. Each Asset now has its own Data Streams price account, derived from its mint and the feed id, so two Assets on one feed no longer share a stored report.

© 2026 Adevar Labs. All rights reserved.

L05: Two Assets on one feed can flip the stored scale



LOCATION

reflect-proxy-program/9819f6e6/program/src/proxy/post_oracle_price.rs:L370-L378

RELEVANT SNIPPET

>_ post_oracle_price.rs RUST

```
368:  }
369:
370:  stored.set_inner(
371:      *feed_id,
372:      price.price,
373:      price.exponent,
374:      price.confidence_bps.unwrap_or(u16::MAX),
375:      report.observations_timestamp,
376:      now.slot,
377:      self.instruction_data.bump,
378:  );
379:
380:  Ok(())
```

DESCRIPTION

A **StreamPrice** account is one PDA per feed id, shared by every Asset that names that feed. **post_oracle_price** is permissionless, and the caller chooses which Asset to pass. The exponent written into the account is **-decimals** from that Asset, not a scale from the report.

load_stream_price accepts the account only when that stored exponent matches the leg that is reading. If one Asset says 8 and another says 18, the one that did not post last gets **OracleScaleMismatch** and cannot price. The price is not used at the wrong scale.

Taking the account back requires a report with a strictly newer **observationsTimestamp**, posted through the Asset with the right decimals. The other Asset can flip the exponent again on the next report. This happens only when two Assets on the same feed disagree on **decimals**.

RECOMMENDATION

When more than one Asset names a feed, require them to use the same **decimals** before a post is accepted. A post through a different scale should fail instead of rewriting the stored exponent.

© 2026 Adevar Labs. All rights reserved.

FIX REVIEW NOTES

Fixed in **448213c**. With a Data Streams price account per Asset, two Assets with different **decimals** can no longer rewrite each other's stored scale. A post at a different scale is refused with **OracleScaleMismatch**, a check added in **8b71e4e** during the review.

© 2026 Adevar Labs. All rights reserved.

L06: A future observationsTimestamp can freeze a Data Streams feed



LOCATION

reflect-proxy-program/9819f6e6/program/src/stream_report.rs:L232-L234

RELEVANT SNIPPET

>_ stream_report.rs — RUST

```
230: // old. `valid_from_timestamp` has already refused that case with an accurate error.
231: let bound_seconds = staleness_bound_seconds(max_staleness) as i64;
232: if now_unix_timestamp.saturating_sub(self.observations_timestamp as i64) > bound_secon
       ds {
233:     return Err(ProxyError::StaleOracle.into());
234: }
235:
236: if self.price <= 0 || self.bid < 0 {
```

DESCRIPTION

validate checks that **now** is inside **validFromTimestamp** and **expiresAt**. It does not check that **observationsTimestamp** is at or before **now**. The staleness check subtracts with saturation, so a future observation looks fresh instead of not-yet-valid. **validFromTimestamp** does not cover this, because it is a different field.

The same saturation is in **check_publish_time_staleness**, so **wrap** will use the stored price until **posted_slot** goes stale. A later honest report is refused when its observation is less than or equal to the stored one:

RUST

```
// program/src/proxy/post_oracle_price.rs
if report.observations_timestamp <= stored.observations_timestamp() {
    return Err(ProxyError::OracleReportNotNewer.into());
}
```

Equality is refused too, so a keeper cannot post the same report again just to refresh **posted_slot**. The feed stays unusable until a DON report arrives with a higher **observationsTimestamp**, or the authority points the Asset at a new feed id. A new feed id is a new PDA. There is no admin clear for the old one.

This needs a report the DON signed. A timestamp a few seconds ahead heals on the next higher observation. A timestamp far ahead, while **validFromTimestamp** and **expiresAt** still contain **now**, does not.

© 2026 Adevar Labs. All rights reserved.

PRACTICAL IMPACT

One signed report with a future observation can stick that feed's price, and then block every later report that is not even further ahead. **wrap** uses the future price until the posting slot goes stale, and after that the feed cannot be refreshed. Worst case, the feed stays dead until the DON publishes a later timestamp or the authority moves the Asset to a new feed id.

RECOMMENDATION

In **validate**, require **observationsTimestamp <= now**. Keep the existing rule that a new report must be strictly newer than the stored one.

FIX REVIEW NOTES

Fixed in **8b71e4e**, during the review. Both Data Streams report schemas now refuse a report whose **observationsTimestamp** is ahead of the clock, and **b7ca56e** applies the same rule to Pyth posts.

© 2026 Adevar Labs. All rights reserved.

L07: Opposite inverse flags on one feed compose to $1.00



LOCATION

reflect-proxy-program/f7e2d7fd/program/src/helpers.rs:L394-L398

RELEVANT SNIPPET

>_ helpers.rs — RUST

```
392:  }
393:
394:  acc = if *inverse {
395:      mul_div_floor(acc, COMPOSITION_SCALE, scaled)?
396:  } else {
397:      mul_div_floor(acc, scaled, COMPOSITION_SCALE)?
398:  };
399:
400:  if let Some(leg_bps) = leg.confidence_bps {
```

DESCRIPTION

update_oracle stores the legs the authority sends but it does not compare their oracle addresses or their feed ids.

compose_oracle_legs multiplies by a straight leg and divides by an inverse leg.

Use one feed whose price is $76,822. The authority sets leg 0 to that feed with **inverse = 0**. The authority sets leg 1 to the same feed with **inverse = 1**. The second leg divides the first and the composed price is **10^18** with exponent **-18**.

The band divides that price by **10^12**. The result is **1_000_000**, which is $1.00. A band from **1_000_000** to **1_200_000** accepts the chain.

Each leg adds its confidence. The default chain bound is 200 basis points. Two copies of a feed inside 100 basis points stay inside that bound.

RECOMMENDATION

Reject two legs that share an oracle address and use different inverse flags. Additionally, reject two legs that share a feed id and use different inverse flags also.

FIX REVIEW NOTES

Fixed in **2b9fd31**. Saving an oracle chain now fails when two legs share an oracle address or a feed id and carry opposite **inverse** flags.

© 2026 Adevar Labs. All rights reserved.

5. Enhancement Opportunities

E01: claim_fees takes no signer, so any caller picks the price at which commission settles

LOCATION

reflect-proxy-program/8b71e4ef/program/src/proxy/claim_fees.rs:L63-L92

RELEVANT SNIPPET

>_ claim_fees.rs RUST

```rust
type Error = ProgramError;

fn try_from(accounts: &'a [AccountInfo]) -> Result<Self, Self::Error> {
    let [authority, stablecoin_authority_token_account, proxy_state, stablecoin_proxy_state_vault, oracle, asset, token_program, stablecoin_mint, extra_oracles @ ..] =
        accounts
    else {
        return Err(ProgramError::NotEnoughAccountKeys);
    };

    // Account Checks
    if !proxy_state.is_writable() {
        return Err(ProxyError::NotMutable.into());
    }

    // token_program.key() is used as a seed for the canonical vault ATA derivation and the SPL
    // CPIs hardcode the classic SPL Token program, so the only supported value is Tokenkeg.
    // Assert it explicitly to return a clear error instead of a misleading address mismatch or
    // an opaque CPI failure if a different token program (e.g. Token-2022) is passed.
    if token_program.key() != &pinocchio_token::ID {
        return Err(ProxyError::InvalidTokenProgram.into());
    }

    let proxy_state_data = ProxyState::load(proxy_state)?;
    let stablecoin_authority_token_account_data =
        TokenAccount::from_account_info(stablecoin_authority_token_account)?;

    if stablecoin_authority_token_account_data
        .owner()
        .ne(proxy_state_data.authority())
    {
        return Err(ProxyError::InvalidTokenOwner.into());
    }

    let (derived_stablecoin_proxy_state_vault, _) = find_program_address(
```

OTHER LOCATIONS

- reflect-proxy-program/8b71e4ef/program/src/state.rs:L1455-L1493

© 2026 Adevar Labs. All rights reserved.

DESCRIPTION

ClaimFeesAccounts::try_from never checks **authority**. **claim_fees** pays **vault_amount - ceil(principal / price)**, then calls **empty_integrators_commission**. Any caller picks the price.

At a peak, the claim removes the commission margin above the **VaultUndercollateralized** guard in **unwrap**.

At a dip, the claim pays less than the booked commission and erases the rest. On recovery, **crank** books that value as interest and moves (1 - fee) of it to **principal**.

POTENTIAL BENEFIT

Only the integrator chooses when commission settles. A dip claim no longer costs $448 of a $5,000 commission (1,000,000-token vault, 4.8 bp dip, 10% fee).

RECOMMENDATION

Require the proxy authority to sign **claim_fees**, as **update_proxy** does. Reduce **integrators_commission** by the USD value paid instead of zeroing it. The second change also protects honest claims at a dip.

RUST

```rust
// ClaimFeesAccounts::try_from
if !authority.is_signer() {
    return Err(ProxyError::NotSigner.into());
}
if proxy_state_data.authority().ne(authority.key()) {
    return Err(ProxyError::InvalidProxyAuthority.into());
}

// ProxyState::claim_fees, in place of empty_integrators_commission()
let paid_usdc = u64::try_from(
    mul_oracle(usdc_plus_value_of_commissions as u128, oracle_price, exponent)
        .ok_or(ProgramError::ArithmeticOverflow)?,
)
.map_err(|_| ProgramError::ArithmeticOverflow)?;
self.set_integrators_commission(self.integrators_commission().saturating_sub(paid_usdc));
```

FIX REVIEW NOTES

Fixed in **07c6386** and **98990fa**. **claim_fees** now requires the proxy authority to sign, and it reduces the commission by the value actually paid instead of zeroing it.

© 2026 Adevar Labs. All rights reserved.

E02: SDKs still encode the pre-PR Asset layout and instruction data

LOCATION

reflect-proxy-program/8b71e4ef/sdk/ts/src/generated/accounts/asset.ts:L54-L62

RELEVANT SNIPPET

>_ asset.ts TYPESCRIPT

```
52: }
53:
54: export function getAssetDecoder(): FixedSizeDecoder<Asset> {
55:   return getStructDecoder([
56:     ['stablecoinMint', getAddressDecoder()],
57:     ['oracle', getAddressDecoder()],
58:     ['active', getU8Decoder()],
59:     ['bump', getU8Decoder()],
60:     ['feedId', getAddressDecoder()],
61:   ]);
62: }
63:
64: export function getAssetCodec(): FixedSizeCodec<AssetArgs, Asset> {
```

DESCRIPTION

getAssetDecoder in the TS SDK and **Asset** in the Rust SDK still read the pre-PR 98-byte layout. The program now stores 259 bytes with **legs** at offset 40. On a migrated Asset, the SDK reads **active** from byte 64, inside **legs[0].oracle**.

createAsset still encodes 65 bytes and 5 accounts. The program requires 81 bytes and a sixth **oracle** account. **updateOracle** encodes 64 bytes, not **73 * n + 18**. No **migrateAsset** builder exists.

POTENTIAL BENEFIT

Clients decode Asset accounts correctly, and **createAsset**, **updateOracle** and **migrateAsset** calls built with the SDK reach the program.

RECOMMENDATION

Regenerate both SDKs from the PR head through **codama.ts**, including the new **Asset** layout and a **migrateAsset** instruction.

FIX REVIEW NOTES

The SDKs are regenerated with the new Asset layout and a **migrateAsset** instruction on the **feat/oracle-stacking-sdk-v7** branch (**b8b7e81**), which has not been merged into **main**.

© 2026 Adevar Labs. All rights reserved.

E03: The per-leg confidence bound check runs against non-Pyth legs

LOCATION

reflect-proxy-program/f7e2d7fd/program/src/proxy/update_oracle.rs:L240-L248

RELEVANT SNIPPET

>_ update_oracle.rs RUST

```rust
        value => value,
};
for leg in legs[..leg_count].iter() {
    let effective_leg = match leg.max_confidence_bps {
        0 => DEFAULT_MAX_CONFIDENCE_BPS,
        value => value,
    };
    if effective_leg > effective_chain {
        return Err(ProxyError::InvalidOracleConfiguration.into());
    }
}

Ok(Self {
```

DESCRIPTION

update_oracle compares every leg's effective confidence bound against the chain bound. A leg with **max_confidence_bps == 0** uses **DEFAULT_MAX_CONFIDENCE_BPS = 200**. Chainlink and Doppler legs must have **max_confidence_bps == 0**, because the parser rejects a non-zero bound on a provider that publishes no confidence.

The effective per-leg bound for a Chainlink or Doppler leg is therefore always 200 bps. Any chain that includes such a leg then requires **max_composed_confidence_bps >= 200**. The current code refuses **max_composed_confidence_bps = 100** on a mixed Pyth-plus-Chainlink chain, even though the Chainlink leg contributes nothing to the composed confidence sum.

POTENTIAL BENEFIT

The chain confidence bound then reflects only the legs that publish confidence. An operator can set a tight chain bound on a mostly-Pyth chain even when a Chainlink or Doppler hop rides along.

RECOMMENDATION

Inside the per-leg loop, read the leg's provider tag. Skip past legs whose provider does not publish confidence. Only Pyth legs then feed the per-leg-vs-chain comparison.

FIX REVIEW NOTES

Fixed in **8b71e4e**, during the review. The per-leg check in **update_oracle** now skips legs whose provider publishes no confidence, so only Pyth and Data Streams price legs are compared against the chain bound.

© 2026 Adevar Labs. All rights reserved.

E04: Chainlink staleness checks slot and skips Transmission.timestamp

LOCATION

reflect-proxy-program/f7e2d7fd/program/src/helpers.rs:L720

RELEVANT SNIPPET

>_ helpers.rs RUST

```rust
let answer = i128::from_le_bytes(data[CL_T_ANSWER..CL_T_ANSWER + 16].try_into().unwrap
  ());

check_slot_staleness(slot, max_staleness)?;

if answer <= 0 {
```

DESCRIPTION

A Chainlink **Transmission** contains both a slot and a Unix timestamp. In the supported layout, the slot starts at byte 200 and the timestamp at byte 208. **load_chainlink_price** checks the slot for staleness but does not read the timestamp.

The slot check fits the current trust model: Chainlink's OCR store writes the round, and a wrap caller cannot give an old round a new slot. This is different from Pyth, where an old signed update can be posted to an account at a recent slot.

The timestamp could still provide a useful second check. If a round has a recent slot but an unexpectedly old timestamp, the current reader would accept it. The code already has **check_publish_time_staleness**, which applies a leg's configured staleness bound to Unix time.

POTENTIAL BENEFIT

Chainlink and Pyth then use the same two clocks: slot and unix time. A round whose wall clock is old fails even when the slot still looks fresh.

RECOMMENDATION

Read the **u32** timestamp at byte 208, reject zero, and pass it as an **i64** to **check_publish_time_staleness** after the slot check. This adds a second freshness check without changing how the oracle account is selected or trusted.

FIX REVIEW NOTES

Fixed in **9819f6e**, during the review. **load_chainlink_price** now also checks **Transmission.timestamp** against the leg's bound in seconds.

© 2026 Adevar Labs. All rights reserved.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in Web3 audits.

The firm was built by a mix of experienced professionals from traditional enterprise security and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members whose publications have earned over 1,100 academic citations, and with the trust of projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the Web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is designed to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Finally, we examine external integrations with other Solana projects and verify that inputs and return values are handled properly.

© 2026 Adevar Labs. All rights reserved.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited, we may also include a detailed proof-of-concept exploit with code snippets and instructions on how the attack could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

© 2026 Adevar Labs. All rights reserved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, express or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time of the engagement.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

© 2026 Adevar Labs. All rights reserved.